PRODUCTION MASTERPLAN

Git & GitHub Masterclass: YouTube Creator Blueprint

Complete Production Pipeline, Visual Rig Setup, Pedagogy, and Industry Workflow Curriculum

COURSE DURATION	TARGET AUDIENCE	TARGET RETENTION	PRODUCTION SCOPE
100–120 Mins	**All Developers**	**46%+ AVD**	**3–4 Weeks**
Single Masterclass or 5-Part Series	Beginners, CS students & professionals	Visual tree animations prevent drop-off	Conflict simulation to publishing

4-Week Video Production & Delivery Timeline

EXECUTION WORKFLOW

Phase	Focus Area	Key Actions & Deliverables	Milestone
Week 1	Pedagogy & Demo Sandbox	Architect dual-user demo (simulating 2 devs on GitHub with conflicting branches). Draft visual commit tree storyboards. Build companion repo.	Curriculum Locked
Week 2	Animation & Asset Prep	Design dynamic DAG (Directed Acyclic Graph) commit animations. Configure dual terminal windows for developer A and developer B.	Visual Decks Ready
Week 3	Studio Screencast Capture	Record clean 1440p 60fps screencasts. Capture Git Graph visualizer in VS Code alongside interactive terminal commands (`git log --oneline --graph`).	A-Roll & B-Roll Locked
Week 4	Editing, Packaging & Launch	Add zoom cuts on merge conflict markers (`<<<<<<< HEAD`), generate 3 click-worthy thumbnails, and deploy public starter template.	Published & Promoted

Recording Environment, Terminal Setup & Retention Hooks

PRODUCTION QUALITY

Visual & Terminal Screen Standard

- **Visual Git Extensions:** Keep VS Code Git Graph or GitKraken open in side pane to render real-time tree changes.
- **CLI Typography:** JetBrains Mono at 19pt–21pt; use Git log aliases (`git lg`) with color-coded branch pointers.
- **Dual-User Persona:** Use distinct terminal color schemes for Dev 1 (Alice - Blue) and Dev 2 (Bob - Green) when simulating team workflows.
- **Keycaster:** Highlight modifier hotkeys during editor commit workflows.

YouTube Retention Engineering

- **First 45 Seconds:** The classic folder nightmare: `final_final_v2_edit.zip` . Frame Git as a true time machine.
- **Fear Removal:** Explicitly state: *"You cannot lose work in Git once committed. I will prove it."*
- **Live Merge Disaster:** Trigger a deliberate, nasty merge conflict and calmly resolve it in 60 seconds.
- **Cheat Sheet Lead Magnet:** Offer a free 1-page PDF printable Git Command Matrix in the pinned comment.

Pedagogical Principle: Snapshots, Not Deltas

Most beginners struggle because they believe Git tracks differences line-by-line. Anchor learning early around the truth: **Git tracks snapshots (SHA-1/SHA-256 hashes of the file tree)**, and branches are simply lightweight movable pointers pointing to a commit hash.

Curriculum Breakdown: Modules 1 to 3

LOCAL GIT ENGINE MASTERY

Module 1: Mental Model, The 3 Trees & First Commits

Timing: 25 Mins

- **Git vs GitHub:** Distributed Version Control System (DVCS) engine vs cloud collaboration platform and CI/CD host.
- **The 3 Stages of Git:** Working Directory (untracked/modified), Staging Area / Index (prepared snapshots), and Repository (`.git` object store).
- **Global Configuration:** Configuring `user.name`, `user.email`, default branch name (`init.defaultBranch main`), and credential managers.
- **The First Snapshot:** Initializing with `git init`, staging with `git add .` vs interactive `git add -p`, and writing clean commits with `git commit -m`.
- **Inspecting History:** Reading `git status`, formatting readable history with `git log --oneline --graph --all`, and `git diff` vs `git diff --staged`.

Module 2: Branching Mechanics, Merging & Conflict Resolution

Timing: 30 Mins

- **What is a Branch?:** Demystifying branch pointers (a 41-byte text file holding a 40-character commit SHA) and the `HEAD` pointer.
- **Branch Switching:** Modern branch navigation with `git switch -c feature` vs legacy `git checkout -b`.
- **Fast-Forward Merges vs 3-Way Merge Commits:** Visualizing commit graphs when linear advancement occurs vs diverging branch histories.
- **Anatomy of a Merge Conflict:** Reading conflict markers (`<<<<<<< HEAD`, `=======`, `>>>>>>> feature`); resolving in VS Code merge editor and completing the merge.
- **The .gitignore Discipline:** Crafting patterns for secrets (`.env`), build outputs (`dist/`), logs, and OS system files.

Module 3: Undoing Mistakes, The Stash & Time Travel

Timing: 25 Mins

- **Fixing the Last Commit:** Modifying commit messages and adding forgotten files cleanly via `git commit --amend --no-edit`.
- **Temporary Context Switching:** Saving work in progress with `git stash save`, listing stashes, applying with `git stash pop`, and inspection.
- **Discarding Changes:** Safely unmodifying files with `git restore <file>` and unstaging with `git restore --staged <file>`.
- **Reset vs Revert:** Safe undo for shared branches using `git revert <SHA>` vs local resets (`--soft`, `--mixed`, `--hard`).
- **The Ultimate Safety Net (git reflog):** Recovering "deleted" commits and abandoned branches after a bad hard reset.

Creator Tip: Live `git reflog` Demonstration

Perform a terrifying `git reset --hard HEAD~3` to wipe recent commits off the screen. Then, calmly invoke `git reflog` and checkout the lost SHA. This single demonstration permanently dissolves student anxiety around breaking their code.

Curriculum Breakdown: Modules 4 & 5

REMOTE COLLABORATION & CI/CD

Module 4: GitHub Cloud, Remotes, SSH & Pull Requests

Timing: 25 Mins

- **Remotes Under the Hood:** Inspecting `git remote -v`; understanding origin tracking branches (`origin/main` vs local `main`).
- **Authentication Setup:** Generating secure SSH keys (`ssh-keygen -t ed25519`), adding to GitHub, and explaining personal access tokens (PATs).
- **Pushing & Synchronizing:** Upstream tracking with `git push -u origin feature`, comparing `git fetch` vs `git pull` (fetch + merge).
- **The Professional Pull Request (PR):** Creating PRs, code review conventions, line-by-line inline commenting, suggested changes, and branch protection rules.
- **Forks vs Clones:** Contributing to open source: Forking upstream repos, maintaining upstream sync, and submitting cross-repo PRs.

Module 5: Advanced Workflows: Rebase, Cherry-Pick & GitHub Actions

Timing: 20 Mins

- **Rebase vs Merge:** Linearizing commit history using `git rebase main`; the golden rule: never rebase public shared branches.
- **Interactive Rebasing:** Cleaning sloppy histories before PR review: squashing 5 micro-commits into 1 clean logical commit with `git rebase -i`.
- **Selective Extraction:** Snatching a specific bugfix from another branch with `git cherry-pick <SHA>`.
- **GitHub Actions Intro:** Creating a basic `.github/workflows/ci.yml` workflow to run linters and automated tests on every pull request.

YouTube Publishing, SEO & Thumbnail Assets

GROWTH & DISTRIBUTION

Titles, Packaging & SEO

• **Title Option 1 (High CTR):** "Git & GitHub Full Course: Beginner to Pro in 2 Hours"

• **Title Option 2 (Fear Relief):** "Stop Fearing Git: The Only Tutorial You Need (2026)"

• **High-Volume Tags:** git tutorial, github crash course, learn git, git commands, merge conflicts, pull request.

Thumbnail Strategy & Growth Assets

• **Thumbnail Layout:** Red Git diamond logo + Octocat + High-contrast visual: Red "MERGE CONFLICT" vs Green "SOLVED".

• **Pinned Comment:** Direct link to GitHub companion repository, chapter timestamps, and PDF cheat sheet.

• **Shorts/Reels Repurposing:** Cut 4 clips: "Undo a commit in 30s", "Git reset vs revert", "What is a branch?", "Resolve conflicts easily".

Creator Action Item: Interactive Practice Repository

Host an intentionally broken public repository with a pre-configured branch conflict. Give viewers a challenge to fork the repo, resolve the conflict, and open a Pull Request. This interactive loop drives massive comment engagement.